# PAGE TURN: LIBRARY MANAGEMENT SYSTEM

A PROJECT REPORT

*Submitted by*

| ROLL NUMBER | REGISTRATION NO. | STUDENT CODE | STUDENT NAME |
|---|---|---|---|
| | | **BWU/MCA/26/016** | **Abhishek Biswas** |

*in partial fulfillment for the award of the degree*

*of*

**MASTER OF COMPUTER APPLICATIONS**

In

DEPARTMENT OF COMPUTATIONAL SCIENCE

BRAINWARE UNIVERSITY

398,Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125



## ABSTRACT

The Library Management System is a web-based application developed to simplify and organize the daily operations of a library. The main purpose of this project is to provide an efficient and user-friendly platform for managing books, student records, book issuance, and book returns.

The application is developed using Python and the Flask web framework. HTML, CSS, and JavaScript are used to design and implement the user interface, while JSON is used for storing and maintaining application data.

The system provides several features, including adding and deleting books, registering and managing students, searching and filtering books, and tracking issued books. It also maintains information about the student who has borrowed a particular book and records the issue date.

The application includes a book return mechanism that calculates the loan duration and applicable late fine. A student is allowed to borrow a book for a period of 14 days, after which a fine of ₹5 per additional day is calculated. The system also prevents the deletion of books that are currently issued and students who have borrowed books.

Furthermore, the dashboard provides an overview of the library through statistical information and graphical representations of book availability and collection details.

This project aims to reduce manual record-keeping, improve data organization, and make library operations more convenient, accurate, and efficient.

# TABLE OF CONTENTS


| Chapter | Title | Page No. |
|---|---|---|
| | **ABSTRACT** | i |
| | **TABLE OF CONTENT** | ii - iii |
| | **LIST OF FIGURES** | iv |
| | **LIST OF TABLES** | v |
| | **LIST OF ABBREVIATIONS** | vi |
| 1 | **INTRODUCTION** | 1 |
| 1.1 | Introduction | 1 |
| 1.2 | Background of the Project | 1 |
| 1.3 | Problem Statement | 1 |
| 1.4 | Proposed Solution | 1 |
| 1.5 | Scope and Importance | 1 |
| 1.6 | Technology Overview | 2 |
| 2 | OBJECTIVES | 3 |
| 2.1 | Primary and Secondary Objectives | 3 |
| 2.2 | Functional and Technical Objectives | 3 |
| 3 | PROJECT PLANNING | 4 |
| 3.1 | Development Methodology | 4 |
| 3.2 | Module Identification | 4 |
| 3.3 | Resource and Risk Planning | 4 |
| 4 | REQUIREMENT ANALYSIS | 5 |
| 4.1 | User and Functional Requirements | 5 |
| 4.2 | Non-Functional Requirements | 5 |
| 4.3 | System and Data Requirements | 6 |
| 4.4 | Existing System and Constraints | 6 |

| | | |
|---|---|---|
| 5 | SYSTEM ANALYSIS AND FLOW | 7 |
| 5.1 | System Overview | 7 |
| 5.2 | Context Level Diagram | 7 |
| 5.3 | Data Flow Diagram – Level 0 | 7 |
| 5.4 | Use Case Diagram | 8 |
| 5.5 | Entity-Relationship Diagram | 9 |
| 5.6 | Book Issue and Return Flowchart | 10 – 11 |
| 6 | PROPOSED SYSTEM DESIGN | 12 |
| 6.1 | System Architecture | 12 |
| 6.2 | Module Design | 13 |
| 6.3 | Data Design and Validation | 13 |
| 7 | IMPLEMENTATION AND EXPERIMENTAL RESULTS | 14 |
| 7.1 | Experimental Setup | 14 |
| 7.2 | Book and Student Management | 14 |
| 7.3 | Book Issue and Return | 15 |
| 7.4 | Search, Filter and Dashboard | 16 |
| 7.5 | Testing and Results | 17 |
| 7.6 | Search and Filter Result | 18 |
| 7.7 | Dashboard and Graphical Result | 19 |
| 7.8 | Performance Observation | 20 |
| 7.9 | Test Summary | 21 |
| 8 | LIMITATIONS AND FUTURE SCOPE | 22 |
| 9 | CONCLUSION | 23 |
| | REFERENCES | 24 |

# LIST OF FIGURES


| Figure No. | Figure Title | Page No. |
|---|---|---|
| Figure 5.1 | Context Level Diagram | 7 |
| Figure 5.2 | Data Flow Diagram – Level 0 | 8 |
| Figure 5.3 | Use Case Diagram | 9 |
| Figure 5.4 | Entity-Relationship Diagram | 10 |
| Figure 5.5 | Book Issue and Return Flowchart | 11 |
| Figure 6.1 | System Architecture Diagram | 12 |
| Figure 7.1 | Main Dashboard of the Library Management System | 14 |
| Figure 7.2 | Book Management Interface | 15 |
| Figure 7.3 | Student Management Interface | 16 |
| Figure 7.4 | Book Issue Interface | 17 |
| Figure 7.5 | Book Return Processing and Fine Calculation | 18 |
| Figure 7.6 | Search and Filter Functionality | 19 |
| Figure 7.7 | Graphical Representation of Library Statistics | 20 |

# LIST OF TABLES


| Table No. | Table Title | Page No. |
|---|---|---|
| Table 4.1 | Functional Requirements | 5 |
| Table 4.2 | Non-Functional Requirements | 5 |
| Table 7.1 | Software Testing Summary | 21 |

# LIST OF ABBREVIATIONS

| Abbreviation | Meaning |
|---|---|
| LMS | Library Management System |
| UI | User Interface |
| HTML | HyperText Markup Language |
| CSS | Cascading Style Sheets |
| JS | JavaScript |
| JSON | JavaScript Object Notation |
| DFD | Data Flow Diagram |
| ERD | Entity-Relationship Diagram |
| CRUD | Create, Read, Update, Delete |
| API | Application Programming Interface |
| HTTP | HyperText Transfer Protocol |

CHAPTER 1

# INTRODUCTION

### 1.1 Introduction

A library provides students and faculty members with access to books and learning resources. Managing books, student records, and issue-return activities manually can be time-consuming. The Library Management System is developed to manage these activities digitally through a web-based application.

### 1.2 Background of the Project

Traditional library management involves maintaining records manually. As the number of books and students increases, tracking records and book availability becomes difficult. This project provides a computerized solution for organizing essential library operations.

### 1.3 Problem Statement

Manual library management may cause the following problems:

- Difficulty in maintaining records.
- Time-consuming book searching.
- Incorrect book availability information.
- Duplicate or incorrect entries.
- Errors in tracking issued books and calculating fines.

### 1.4 Proposed Solution

The proposed system provides a centralized web application with facilities for book management, student registration, book issue and return, searching, filtering, fine calculation, and dashboard statistics.

### 1.5 Scope and Importance

The project focuses on small-scale library management. It helps maintain organized records, track book availability, reduce manual work, and improve the accuracy of routine library operations.

The current system does not include user authentication, barcode scanning, or online reservations.

### 1.6 Technology Overview

The project uses the following technologies:

- **Python and Flask:** Backend development and request processing.
- **HTML and CSS:** Webpage structure and interface design.
- **JavaScript:** Frontend interaction and validation.
- **JSON:** Storage of book and student records.
- **Chart.js:** Graphical presentation of library statistics.

CHAPTER 2

# OBJECTIVES

### 2.1 Primary and Secondary Objectives

The primary objective of this project is to develop a web-based Library Management System to simplify library operations and reduce manual work.

The secondary objectives are:

- To maintain book and student records digitally.
- To simplify book searching and filtering.
- To manage book issuing and returning.
- To update book availability automatically.
- To calculate late-return fines.
- To prevent deletion of records involved in active issues.
- To display library statistics through a dashboard.

### 2.2 Functional and Technical Objectives

**Functional Objectives:** The system provides book management, student registration, book issue and return, search and filter facilities, and dashboard statistics.

**Technical Objectives:** The application uses Python and Flask for backend processing, HTML, CSS, and JavaScript for frontend development, JSON for data storage, and Chart.js for graphical representation.

CHAPTER 3

# PROJECT PLANNING

## 3.1 Development Methodology

The project follows an incremental development approach. The development process includes requirement collection, requirement analysis, system design, implementation, testing, and final documentation.

The application was developed using Python, Flask, HTML, CSS, JavaScript, and JSON.

## 3.2 Module Identification

The system consists of the following modules:

1. **Dashboard:** Displays library statistics and charts.
2. **Book Management:** Adds, searches, and deletes book records.
3. **Student Management:** Registers and manages student records.
4. **Book Issue:** Issues available books to registered students.
5. **Book Return:** Updates book availability and calculates late fines.
6. **Search and Filter:** Helps find books using different fields.
7. **Validation:** Checks input data and prevents invalid operations.

## 3.3 Resource and Risk Planning

**Resources:** A computer, code editor, Python, Flask, web browser, HTML, CSS, JavaScript, JSON, and Chart.js were used.

**Risks and Solutions:**

- Invalid input: Input validation.
- Duplicate records: Unique ID checking.
- Incorrect availability: Automatic status updates.
- Accidental deletion: Restrictions on actively issued records.
- Data inconsistency: Backend validation and structured JSON storage.

CHAPTER 4

# REQUIREMENT ANALYSIS

### 4.1 User and Functional Requirements

The Library Management System is designed to manage books and student records efficiently. The main functional requirements are:

Table 4.1: Functional Requirements

| **Requirement** | **Description** |
|---|---|
| Book Management | Add, search and delete book records. |
| Student Management | Register and manage student details. |
| Book Issue | Issue available books to registered students. |
| Book Return | Return books and update availability. |
| Fine Calculation | Calculate fines for late returns. |
| Search and Filter | Search books using ID, title, author and category. |
| Dashboard | Display book and student statistics. |
| Validation | Prevent invalid and duplicate entries. |

### 4.2 Non-Functional Requirements

Non-functional requirements describe the quality and performance expected from the system.

Table 4.2: Non-Functional Requirements

| **Requirement** | **Description** |
|---|---|
| Usability | Simple and user-friendly interface. |
| Reliability | Maintain accurate book and student records. |
| Performance | Process common operations efficiently. |
| Maintainability | Allow easy modification and updates. |
| Data Integrity | Prevent invalid and inconsistent records. |
| Compatibility | Run through a modern web browser. |

### 4.3 System and Data Requirements

**Hardware Requirements:**

- Computer or laptop.
- Minimum 4 GB RAM.
- Keyboard, mouse and internet browser.

**Software Requirements:**

- Operating System: Windows.
- Programming Language: Python.
- Backend Framework: Flask.
- Frontend: HTML, CSS and JavaScript.
- Data Storage: JSON files.
- Chart Library: Chart.js.
- Code Editor: Visual Studio Code.

The system stores book and student information in separate JSON files. These files are used to maintain and update records.

### 4.4 Existing System and Constraints

Traditional library systems often depend on manual registers or basic record-keeping methods. These methods can be time-consuming and may lead to errors in searching, issuing and returning books.

The proposed system reduces manual work through digital record management. However, it has some constraints:

- Data is stored in JSON files instead of a relational database.
- User authentication and role management are not included.
- Separate historical issue-return records are not maintained.
- Barcode scanning and automatic notifications are unavailable.
- The system is mainly suitable for small-scale library operations.

CHAPTER 5

# SYSTEM ANALYSIS AND FLOW

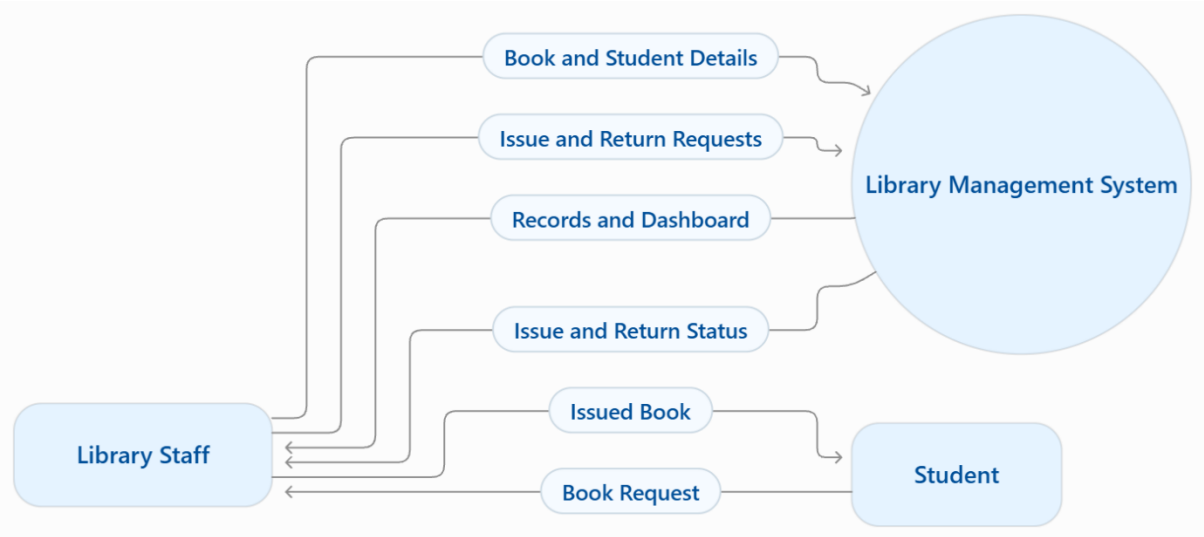
## 5.1 System Overview

The Library Management System is a web-based application developed to manage books, students, issuing, and returning operations. It maintains records using JSON files and provides a dashboard to monitor library activities.

## 5.2 Context Level Diagram

The Context Level Diagram represents the interaction between the Library Management System, library staff, and students.

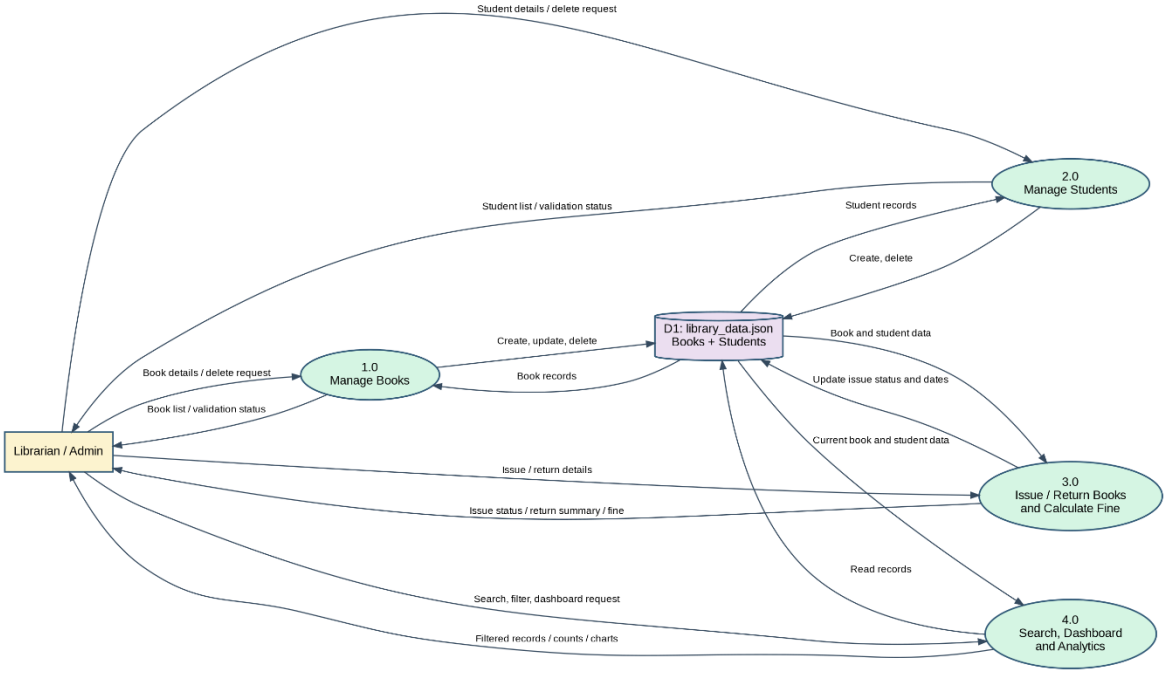
Figure 5.1:Context Level Diagram




## 5.3 Data Flow Diagram – Level 0

The Level 0 DFD shows the main processes of the system, including book management, student management, book issue, book return, and data storage.

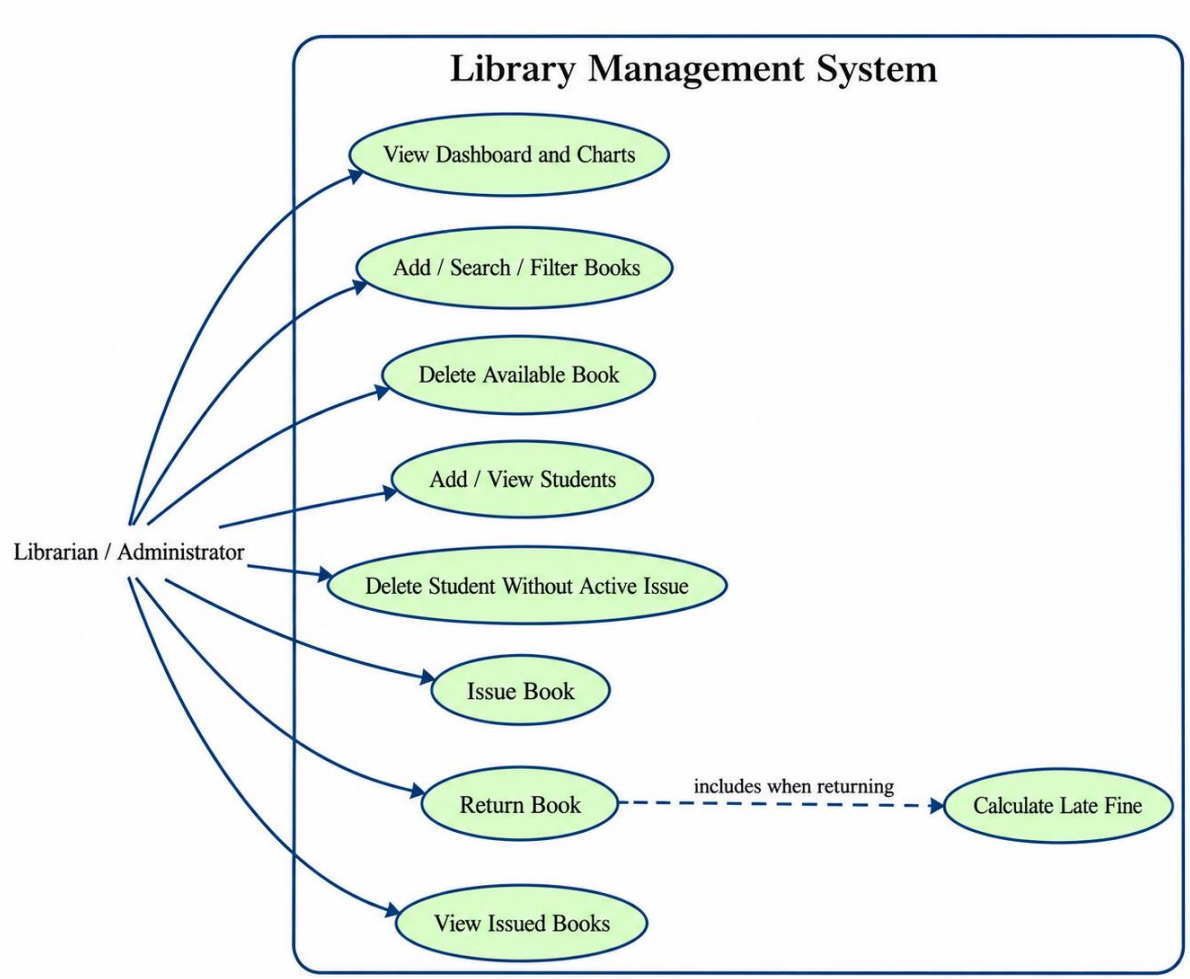
Figure 5.2: Data Flow Diagram – Level 0




## 5.4 Use Case Diagram

The Use Case Diagram represents the activities performed by the library staff, such as managing books, registering students, issuing books, returning books, and searching records.

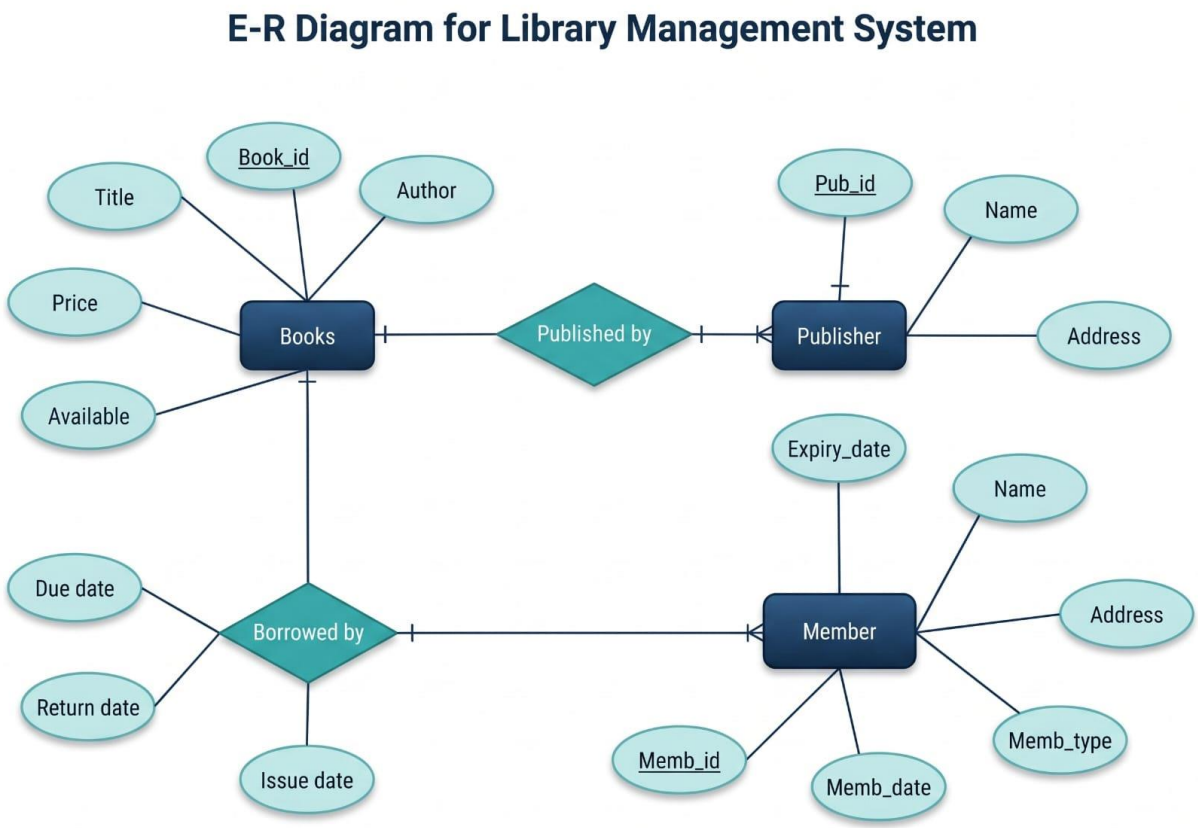
Figure 5.3:Use Case Diagram




### 5.5 Entity-Relationship Diagram

The ER Diagram represents the logical relationship between the Book and Student entities. It shows how book and student records are associated during issue and return operations.

Figure 5.4:Entity-Relationship Diagram

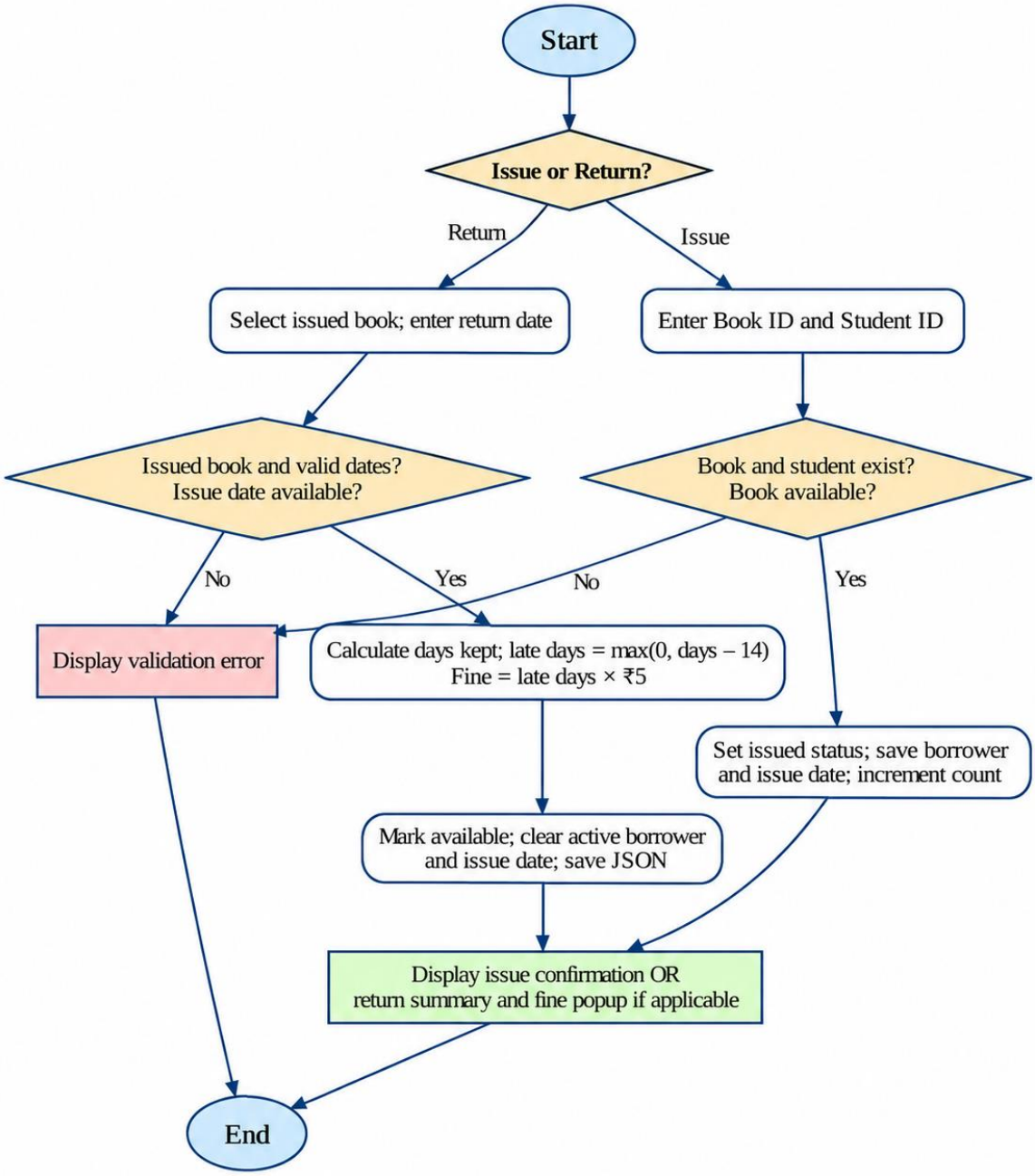



**5.6 Book Issue and Return Flowchart**

The flowchart illustrates the process of checking book availability, issuing books, returning books, updating records, and calculating late-return fines.

Figure 5.5:Book Issue and Return Flowchart

# Book Issue and Return Flowchart



CHAPTER 6

# PROPOSED SYSTEM DESIGN

### 6.1 System Architecture

The Library Management System follows a simple three-layer architecture:

- **Presentation Layer:** Provides the user interface using HTML, CSS, and JavaScript.
- **Application Layer:** Handles application logic and requests using Python and Flask.
- **Data Layer:** Stores book and student records in JSON files.

Chart.js is used to display graphical statistics on the dashboard.

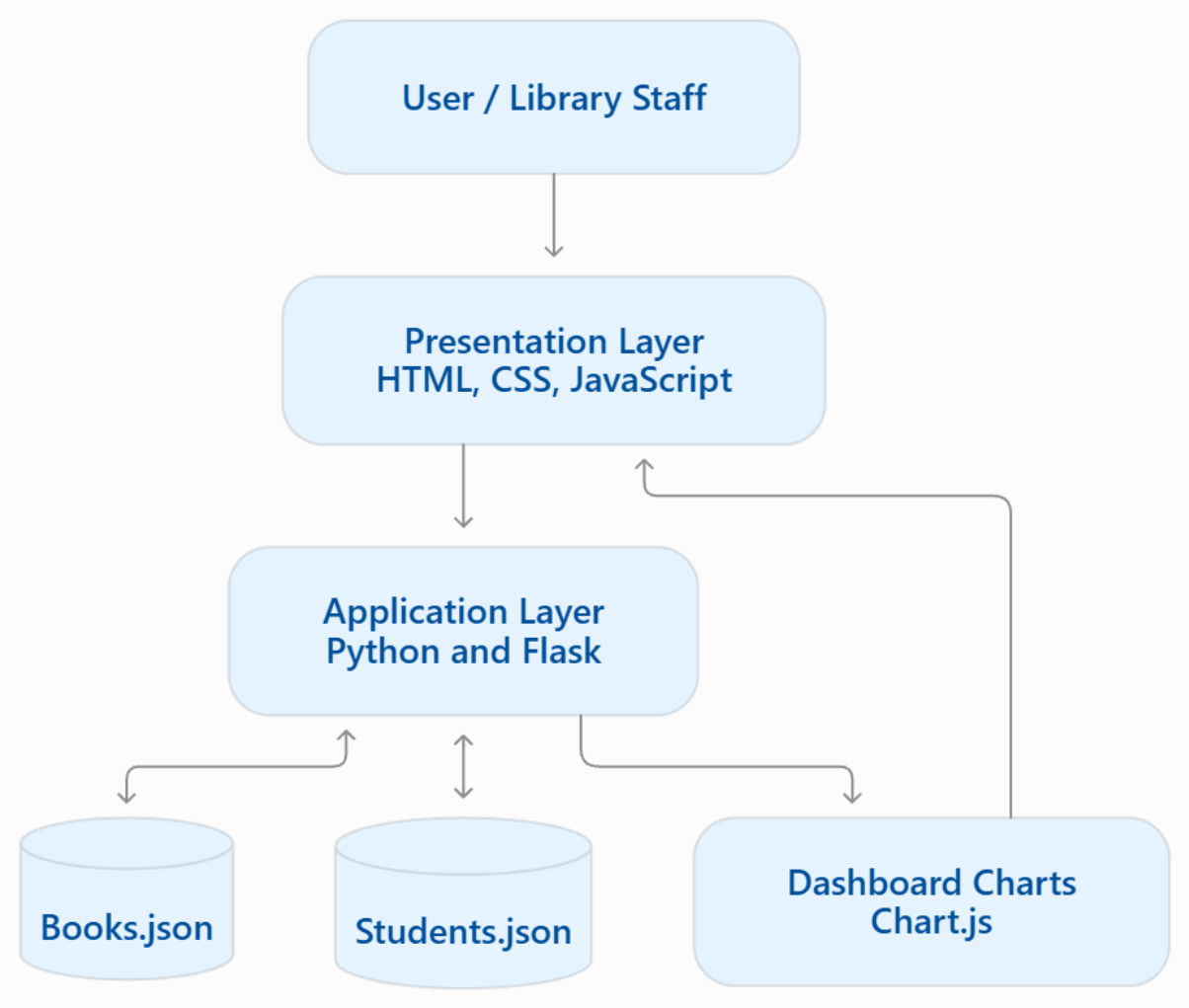



Figure 6.1: System Architecture Diagram

### 6.2 Module Design

The system is divided into the following modules:

1. **Book Module:** Manages book records and availability.
2. **Student Module:** Stores and manages student information.
3. **Issue Module:** Issues available books to registered students.
4. **Return Module:** Updates book status and calculates late fines.
5. **Search Module:** Searches and filters book records.
6. **Dashboard Module:** Displays library statistics and charts.

### 6.3 Data Design and Validation

The system uses two JSON files, Books.json and Students.json, to store records.

Input validation is performed to ensure correct data entry. Duplicate IDs and invalid information are rejected. The system also prevents deleting books or students associated with active book issues.

Book availability and student issue information are updated automatically during issue and return operations.

CHAPTER 7

# IMPLEMENTATION AND EXPERIMENTAL RESULTS

## 7.1 Experimental Setup

The Library Management System was developed using Python, Flask, HTML, CSS, JavaScript and JSON. Chart.js was used for graphical representation. The application was tested on a Windows-based computer using a web browser.

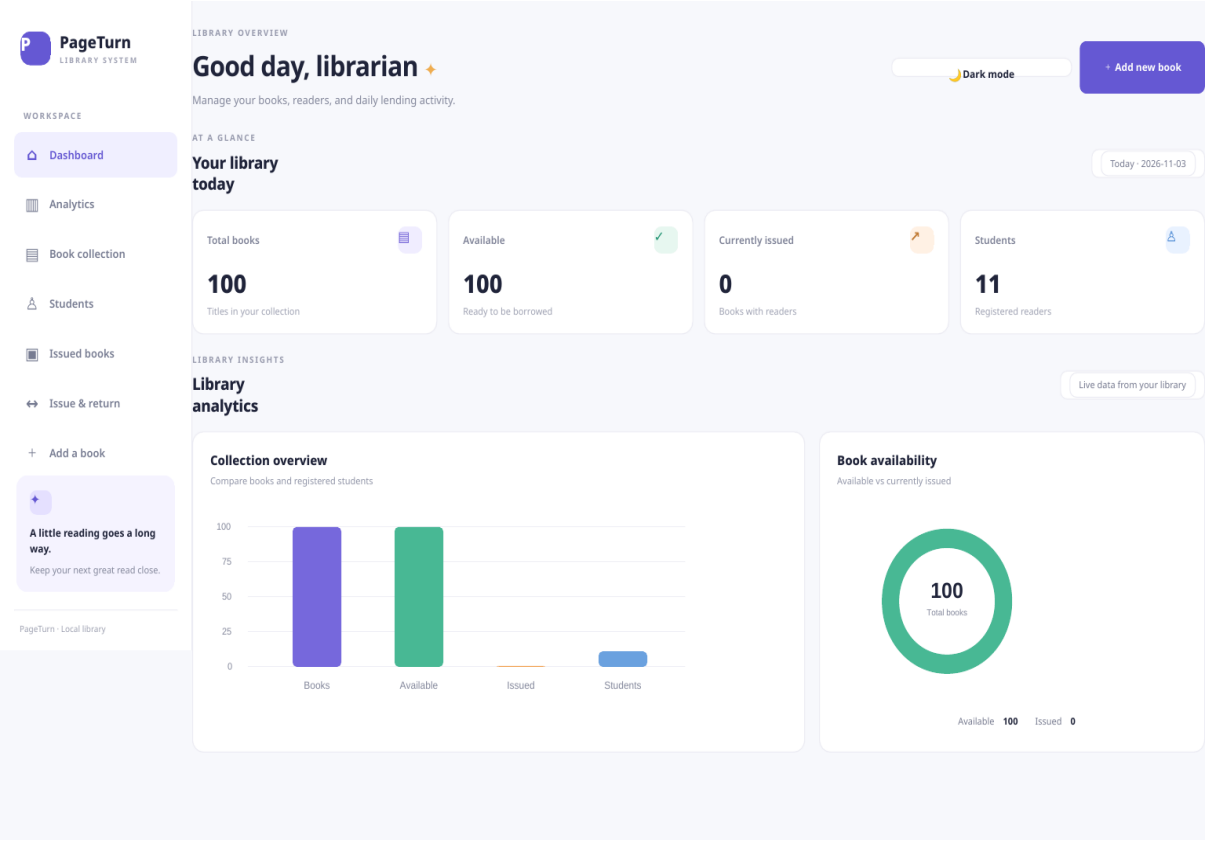



Figure 7.1: Main Dashboard of the Library Management System

## 7.2 Book Management Results

The Book Management module allows users to add, search and delete book records. It also validates book details and prevents duplicate Book IDs.

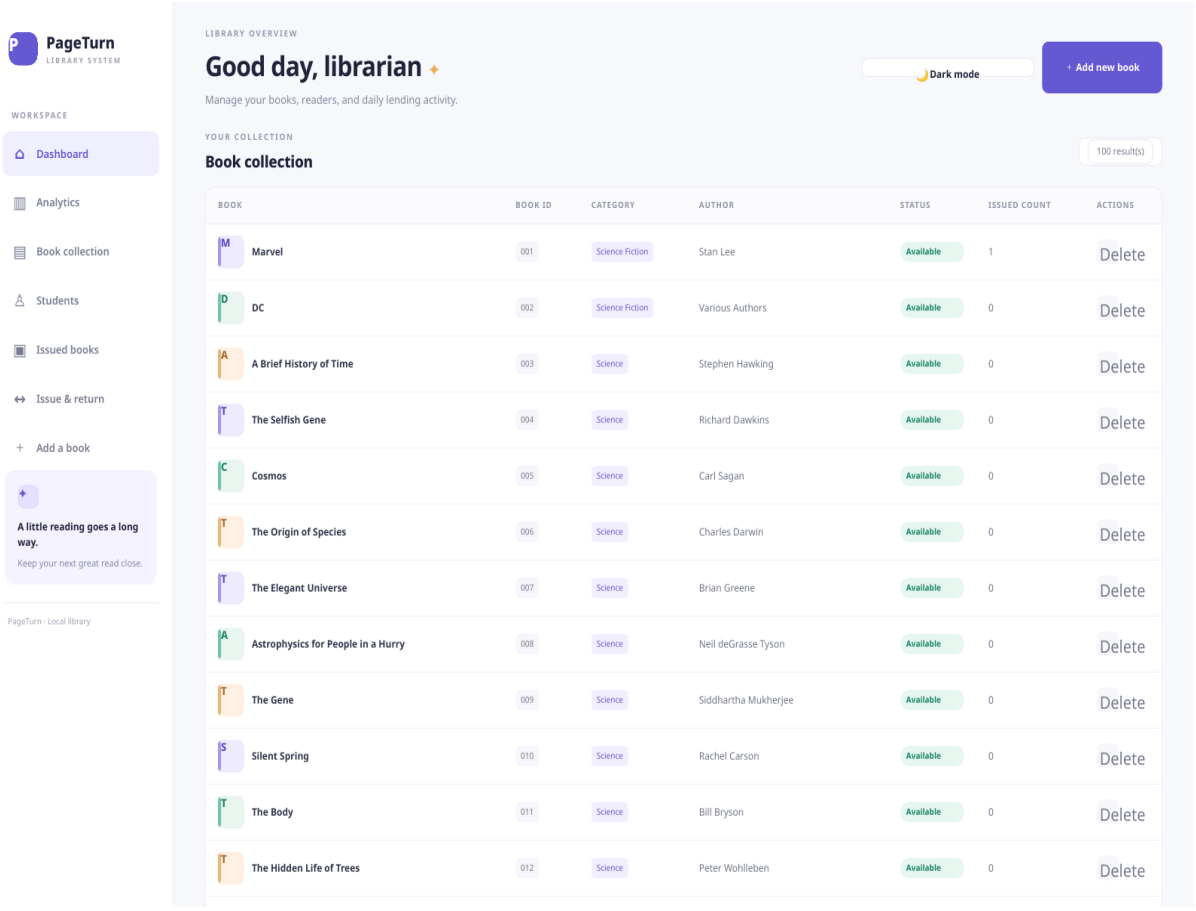



Figure 7.2: Book Management Interface

### 7.3 Student Management Results

The Student Management module allows users to register and manage student details. It validates student information and prevents duplicate Student IDs.

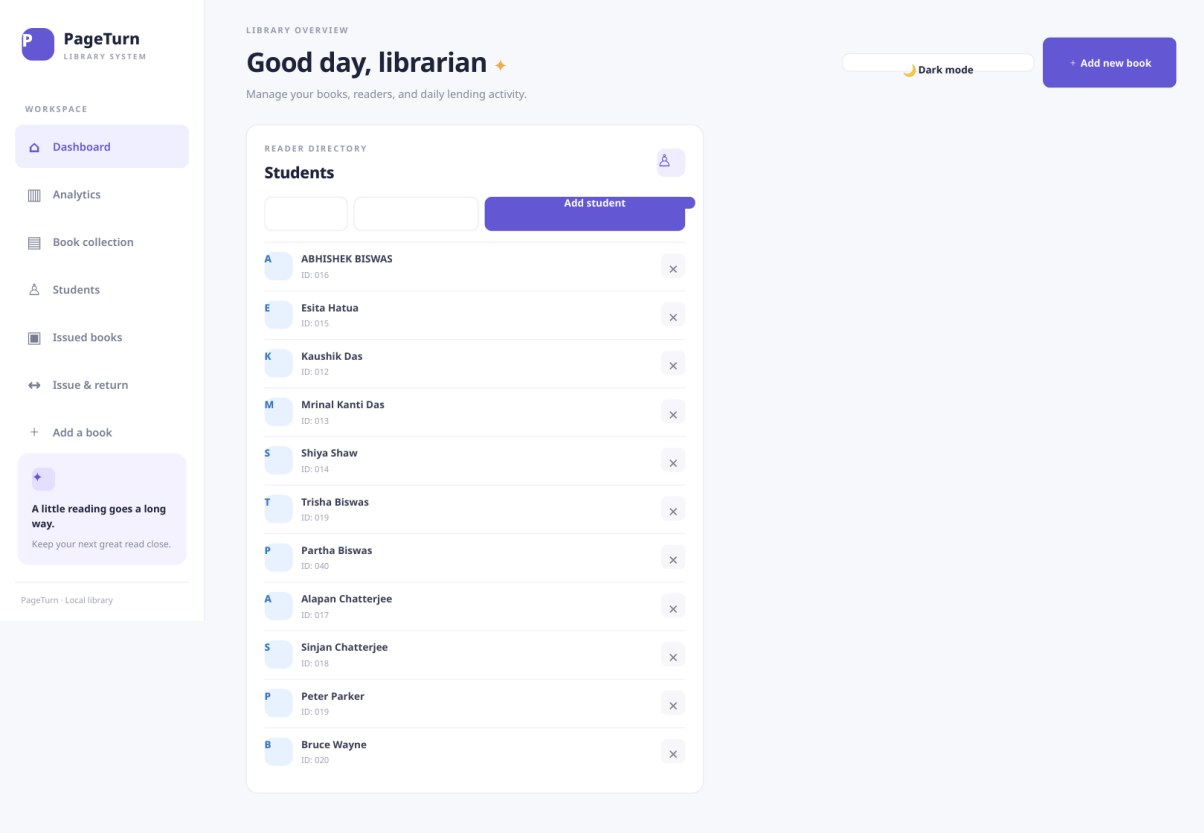



Figure 7.3: Student Management Interface

### 7.4 Book Issue Results

The Book Issue module checks book availability and student records before issuing a book. After successful issue, the book status is updated automatically.

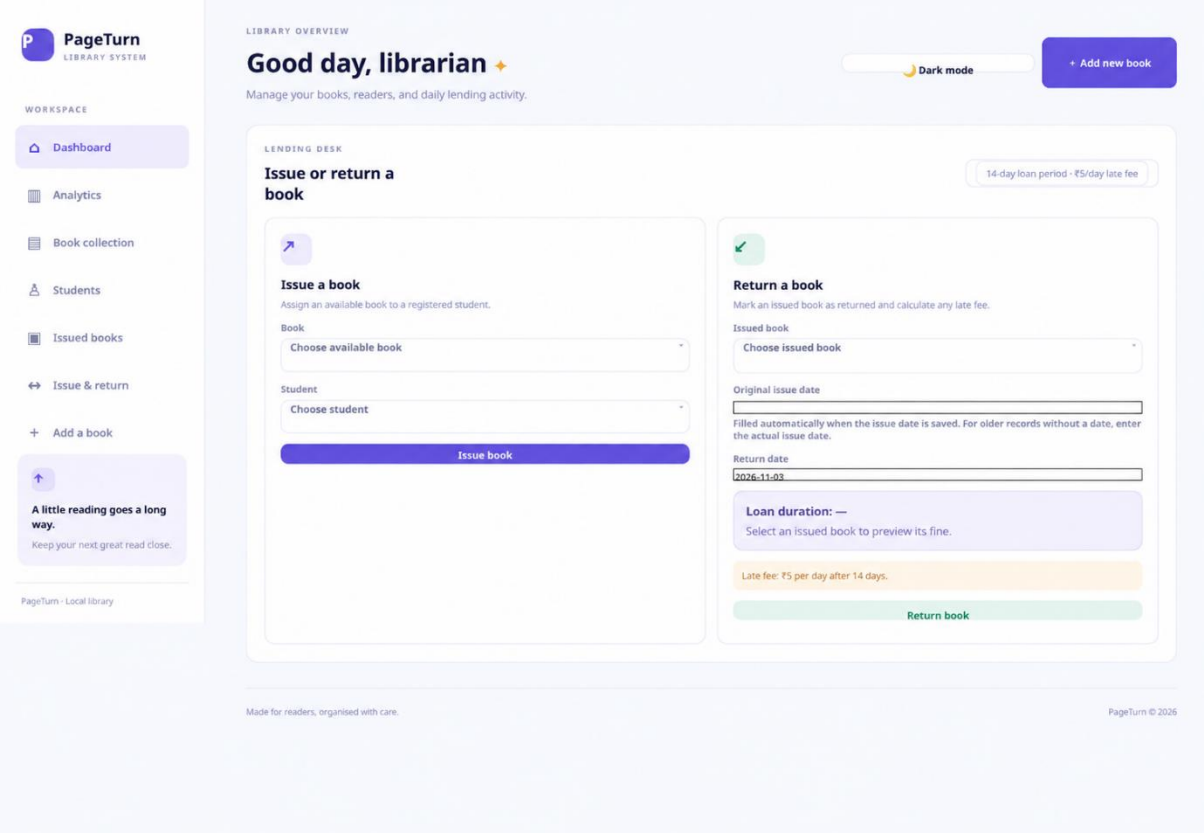



Figure 7.4: Book Issue Interface

### 7.5 Book Return Results

The Book Return module updates book availability after a successful return. A fine of ₹5 per extra day is calculated if the book is returned after the 14-day period.

**Fine = Late Days × ₹5**

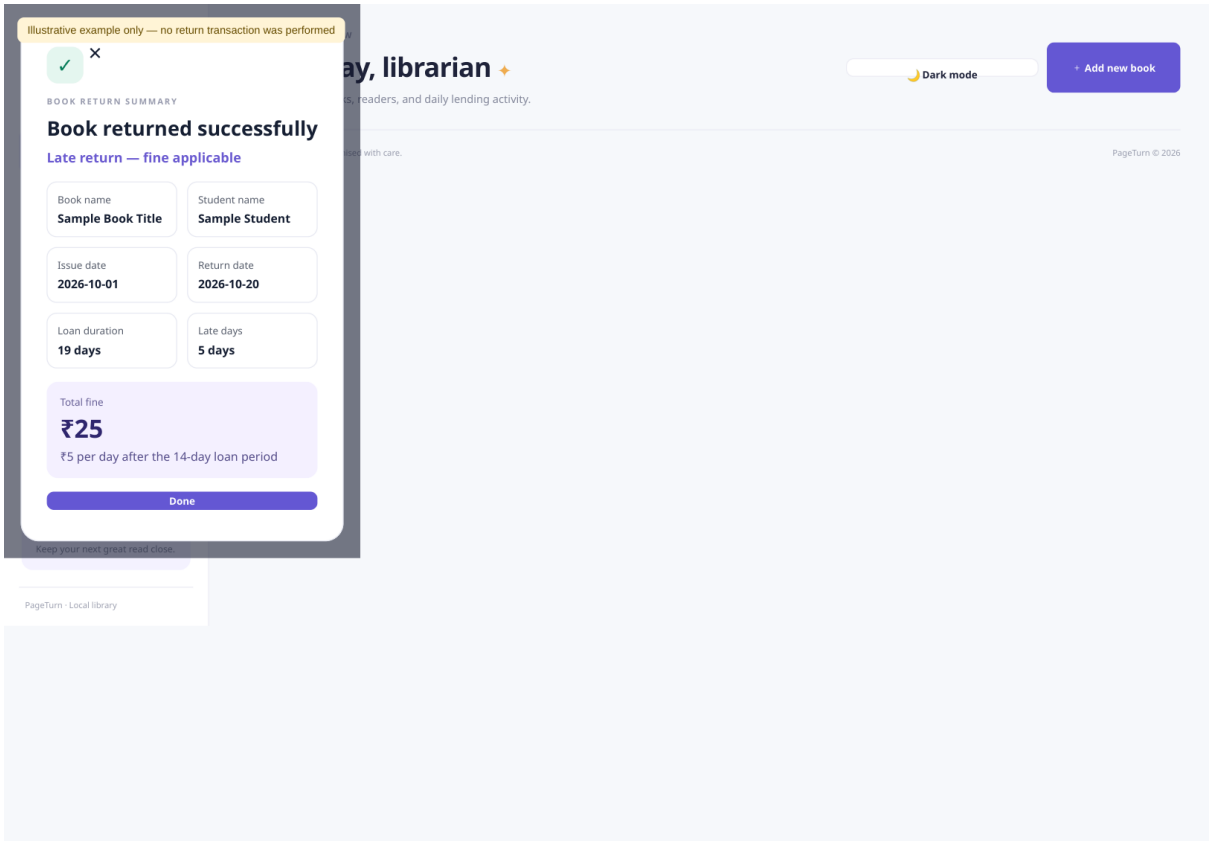



Figure 7.5: Book Return Processing and Fine Calculation

### 7.6 Search and Filter Results

The search and filter facility helps users find books using Book ID, title, author, category and availability.

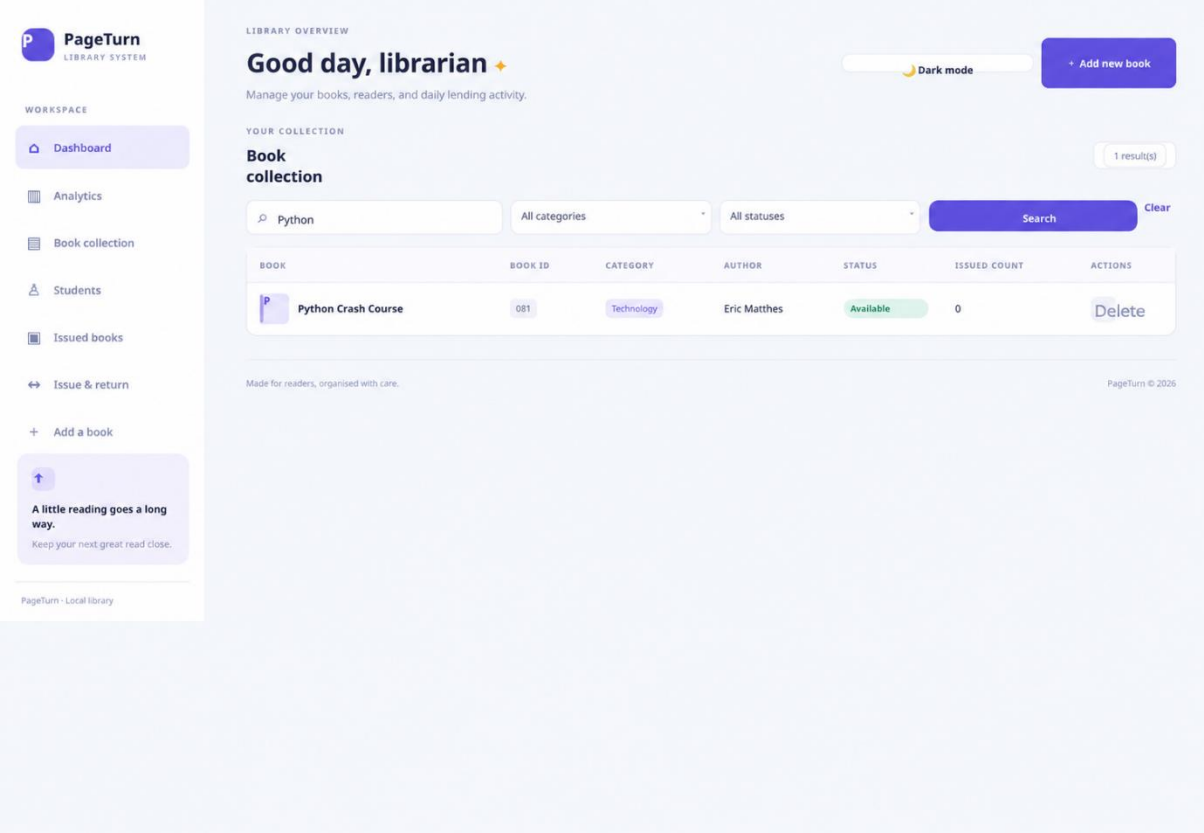



Figure 7.6: Search and Filter Functionality

### 7.7 Dashboard and Graphical Results

The dashboard displays total books, available books, issued books and total students. Chart.js provides graphical representations of library statistics.

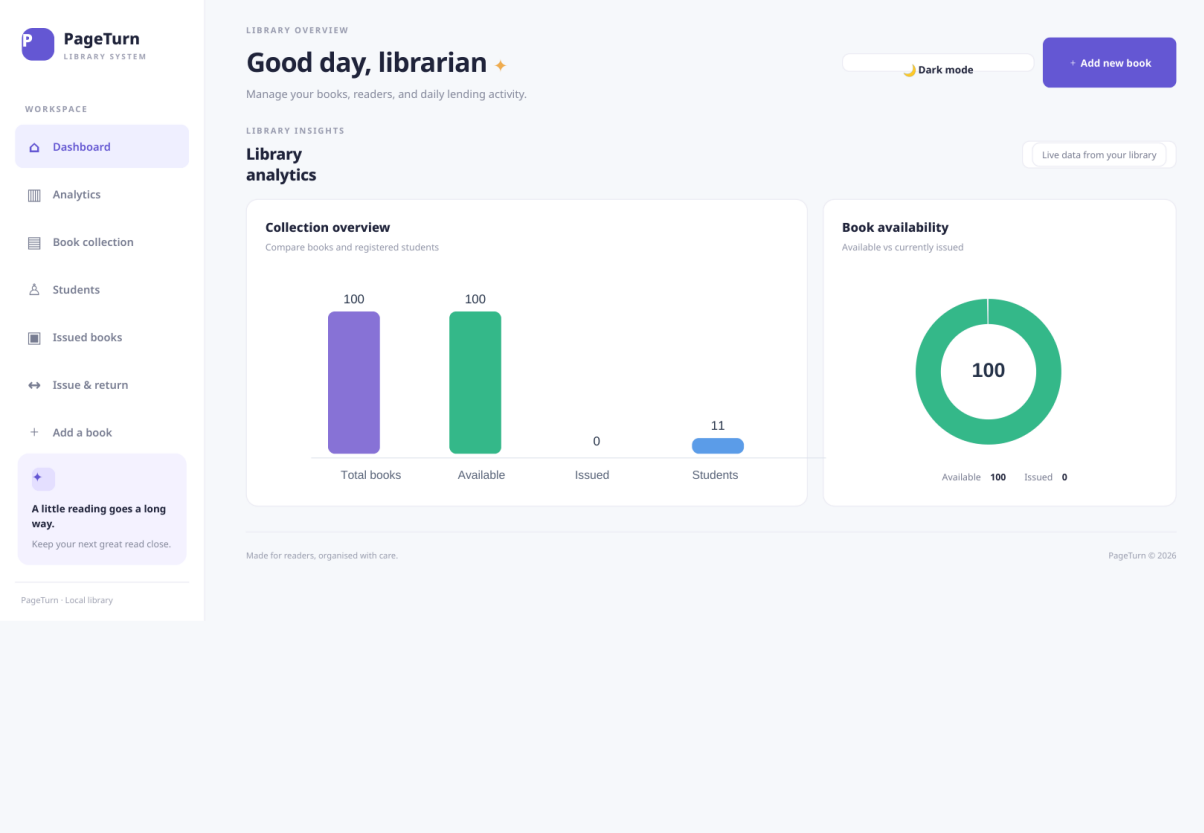



Figure 7.7: Graphical Representation of Library Statistics

### 7.8 Performance Observations

The system performs common library operations efficiently. Input validation and automatic availability updates help maintain accurate records. The simple interface makes library management easier.

### 7.9 Test Summary

The main functions were checked using different test cases to verify their expected behaviour.

**Table 7.1: Software Testing Summary**

| **Test Case** | **Expected Result** |
|---|---|
| Add valid book/student | Record added successfully. |
| Duplicate ID | Entry rejected. |
| Issue available book | Book issued successfully. |
| Issue unavailable book | Operation rejected. |
| Return book | Availability updated. |
| Late return | Fine calculated correctly. |
| Search book | Matching records displayed. |
| Delete active record | Deletion restricted. |

CHAPTER 8

# LIMITATIONS AND FUTURE SCOPE

**Limitations**

The Library Management System has some limitations:

- Data is stored in JSON files instead of a database.
- User authentication and role management are not available.
- Separate issue-return history is not maintained.
- Barcode scanning and automatic notifications are not supported.
- The system is mainly suitable for small-scale libraries.

**Future Scope**

The system can be improved in the future by:

- Integrating a database such as MySQL or MongoDB.
- Adding secure login and user roles.
- Maintaining complete issue-return history.
- Introducing barcode or QR code scanning.
- Adding email and SMS notifications.
- Providing report export and cloud-based access.
- Supporting multiple library branches.

These improvements can make the system more secure, scalable and efficient.

CHAPTER 9

# CONCLUSION

The Library Management System was developed with the aim of simplifying library operations and reducing the difficulties of manual record management. The project provides a digital platform for managing book and student information efficiently.

The system includes important features such as book management, student registration, book issuing and returning, searching and filtering records, and automatic fine calculation for late returns. It also prevents the deletion of books and students involved in active issue records, helping maintain data consistency.

Python and Flask are used for backend processing, while HTML, CSS and JavaScript are used to develop the user interface. JSON files are used for storing book and student records, and Chart.js is used to display graphical information on the dashboard.

The implementation of this project demonstrates how web-based technologies can make library management easier and more organized. It reduces paperwork, saves time, minimizes manual errors and provides quick access to library information. The dashboard also helps users understand the current status of books and students.

Although the system has certain limitations, such as the absence of user authentication, database integration and automatic notifications, these features can be included in future versions to improve its functionality and scalability.

In conclusion, the developed Library Management System fulfils its primary objectives and provides a simple, effective and user-friendly solution for small-scale libraries. It also offers a foundation for further development into a more advanced library management platform.

## REFERENCES

The following resources were used as technical references during the development and documentation of the project.

1. Python Software Foundation, Python Documentation. Available at: https://docs.python.org/3/

2. Pallets Projects, Flask Documentation. Available at: https://flask.palletsprojects.com/

3. MDN Web Docs, HTML: HyperText Markup Language. Available at: https://developer.mozilla.org/en-US/docs/Web/HTML

4. MDN Web Docs, CSS: Cascading Style Sheets. Available at: https://developer.mozilla.org/en-US/docs/Web/CSS

5. MDN Web Docs, JavaScript Documentation. Available at: https://developer.mozilla.org/en-US/docs/Web/JavaScript

6. JSON, Introducing JSON. Available at: https://www.json.org/

7. Chart.js, Chart.js Documentation. Available at: https://www.chartjs.org/docs/

8. W3C, Web Standards and Technical Resources. Available at: https://www.w3.org/